Tarea 1

In [1]:
```
# Importar la base de datos
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
# Importar la base de datos

data = pd.read_csv(r'C:\Users\salasistemas.UTB\Desktop\Especialidad\Business_Analytic
data.head(5)
```

Out[2]:

	ProductionVolume	ProductionCost	SupplierQuality	DeliveryDelay	DefectRate	QualityScor
0	202	13175.403783	86.648534	1	3.121492	63.46349
1	535	19770.046093	86.310664	4	0.819531	83.69781
2	960	19060.820997	82.132472	0	4.514504	90.35055
3	370	5647.606037	87.335966	5	0.638524	67.62869
4	206	7472.222236	81.989893	3	3.867784	82.72833

In [3]:
```
# Sacar numero de filas y columnas de la base de datos
print("Número de filas:", data.shape[0])
print("Número de columnas:", data.shape[1])
```

```
Número de filas: 3240
Número de columnas: 17
```

In [4]:
```
data.info()

# Alternativa más detallada
for col in data.columns:
    print(f"Columna: {col}, Tipo de dato: {data[col].dtype}")
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3240 entries, 0 to 3239
Data columns (total 17 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   ProductionVolume      3240 non-null   int64  
 1   ProductionCost        3240 non-null   float64
 2   SupplierQuality       3240 non-null   float64
 3   DeliveryDelay         3240 non-null   int64  
 4   DefectRate            3240 non-null   float64
 5   QualityScore          3240 non-null   float64
 6   MaintenanceHours      3240 non-null   int64  
 7   DowntimePercentage    3240 non-null   float64
 8   InventoryTurnover     3240 non-null   float64
 9   StockoutRate          3240 non-null   float64
 10  WorkerProductivity    3240 non-null   float64
 11  SafetyIncidents       3240 non-null   int64  
 12  EnergyConsumption     3240 non-null   float64
 13  EnergyEfficiency      3240 non-null   float64
 14  AdditiveProcessTime   3240 non-null   float64
 15  AdditiveMaterialCost  3240 non-null   float64
 16  DefectStatus          3240 non-null   int64  
dtypes: float64(12), int64(5)
memory usage: 430.4 KB
Columna: ProductionVolume, Tipo de dato: int64
Columna: ProductionCost, Tipo de dato: float64
Columna: SupplierQuality, Tipo de dato: float64
Columna: DeliveryDelay, Tipo de dato: int64
Columna: DefectRate, Tipo de dato: float64
Columna: QualityScore, Tipo de dato: float64
Columna: MaintenanceHours, Tipo de dato: int64
Columna: DowntimePercentage, Tipo de dato: float64
Columna: InventoryTurnover, Tipo de dato: float64
Columna: StockoutRate, Tipo de dato: float64
Columna: WorkerProductivity, Tipo de dato: float64
Columna: SafetyIncidents, Tipo de dato: int64
Columna: EnergyConsumption, Tipo de dato: float64
Columna: EnergyEfficiency, Tipo de dato: float64
Columna: AdditiveProcessTime, Tipo de dato: float64
Columna: AdditiveMaterialCost, Tipo de dato: float64
Columna: DefectStatus, Tipo de dato: int64
```

In [5]:

```
print("Valores faltantes por columna:")
print(data.isnull().sum())

print("\nTotal de valores faltantes en el dataset:", data.isnull().sum().sum())

print("\nNúmero de registros duplicados:", data.duplicated().sum())
```

```
Valores faltantes por columna:
ProductionVolume        0
ProductionCost          0
SupplierQuality         0
DeliveryDelay           0
DefectRate              0
QualityScore            0
MaintenanceHours        0
DowntimePercentage      0
InventoryTurnover       0
StockoutRate            0
WorkerProductivity      0
SafetyIncidents         0
EnergyConsumption       0
EnergyEfficiency        0
AdditiveProcessTime     0
AdditiveMaterialCost    0
DefectStatus            0
dtype: int64

Total de valores faltantes en el dataset: 0

Número de registros duplicados: 0
```

In [6]:

```
estadisticas = data.describe().T
estadisticas["mediana"] = data.median(numeric_only=True)
estadisticas = estadisticas[["mean", "mediana", "std", "min", "max"]]
```

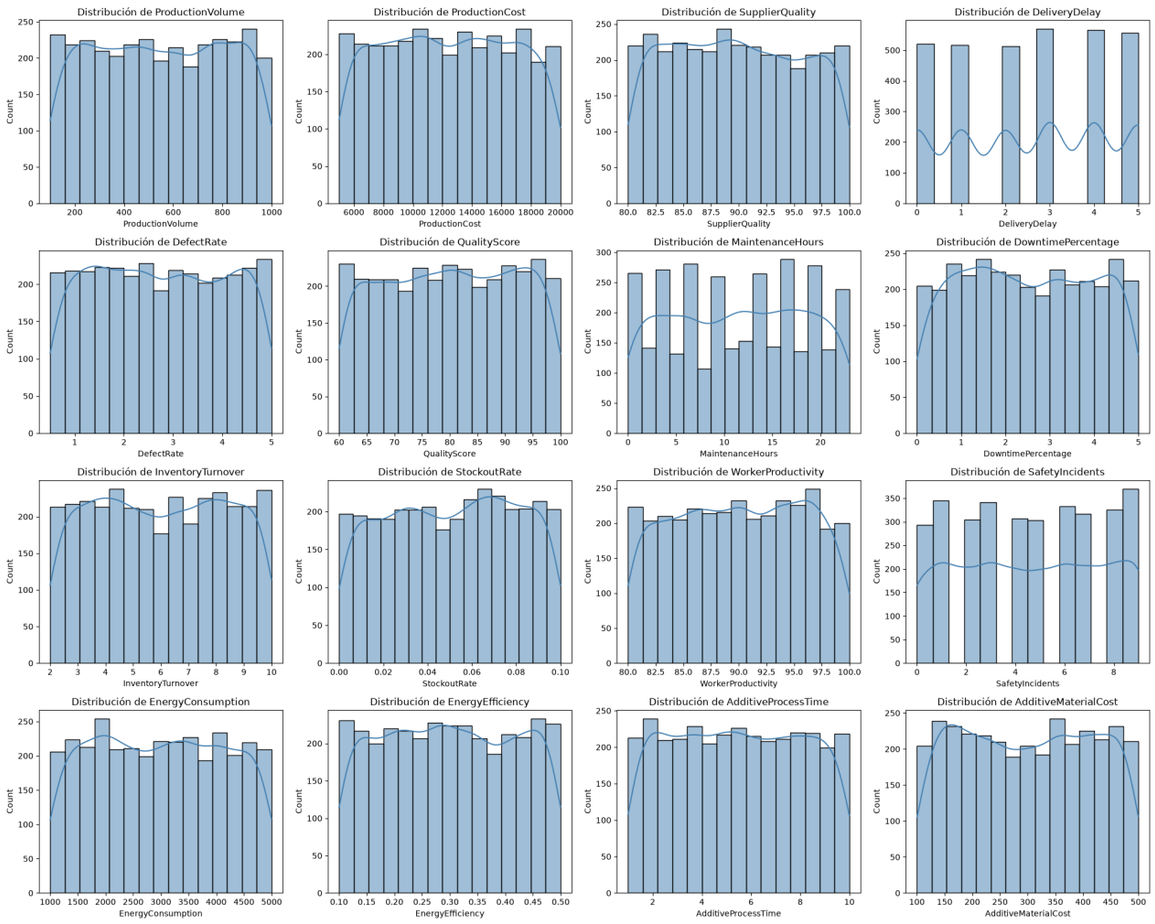
```
estadisticas.columns = ["Media", "Mediana", "Desviación Estándar", "Mínimo", "Máximo"]
estadisticas
```

Out[6]:

	Media	Mediana	Desviación Estándar	Mínimo	Máximo
ProductionVolume	548.523148	549.000000	262.402073	100.000000	999.000000
ProductionCost	12423.018476	12405.204656	4308.051904	5000.174521	19993.365549
SupplierQuality	89.833290	89.704861	5.759143	80.004820	99.989214
DeliveryDelay	2.558951	3.000000	1.705804	0.000000	5.000000
DefectRate	2.749116	2.708775	1.310154	0.500710	4.998529
QualityScore	80.134272	80.265312	11.611750	60.010098	99.996993
MaintenanceHours	11.476543	12.000000	6.872684	0.000000	23.000000
DowntimePercentage	2.501373	2.465151	1.443684	0.001665	4.997591
InventoryTurnover	6.019662	6.022389	2.329791	2.001611	9.998577
StockoutRate	0.050878	0.051837	0.028797	0.000002	0.099997
WorkerProductivity	90.040115	90.125743	5.723600	80.004960	99.996786
SafetyIncidents	4.591667	5.000000	2.896313	0.000000	9.000000
EnergyConsumption	2988.494453	2996.822301	1153.420820	1000.720156	4997.074741
EnergyEfficiency	0.299776	0.297470	0.116400	0.100238	0.499500
AdditiveProcessTime	5.472098	5.437134	2.598212	1.000151	9.999749
AdditiveMaterialCost	299.515479	299.728918	116.379905	100.211137	499.982782
DefectStatus	0.840432	1.000000	0.366261	0.000000	1.000000

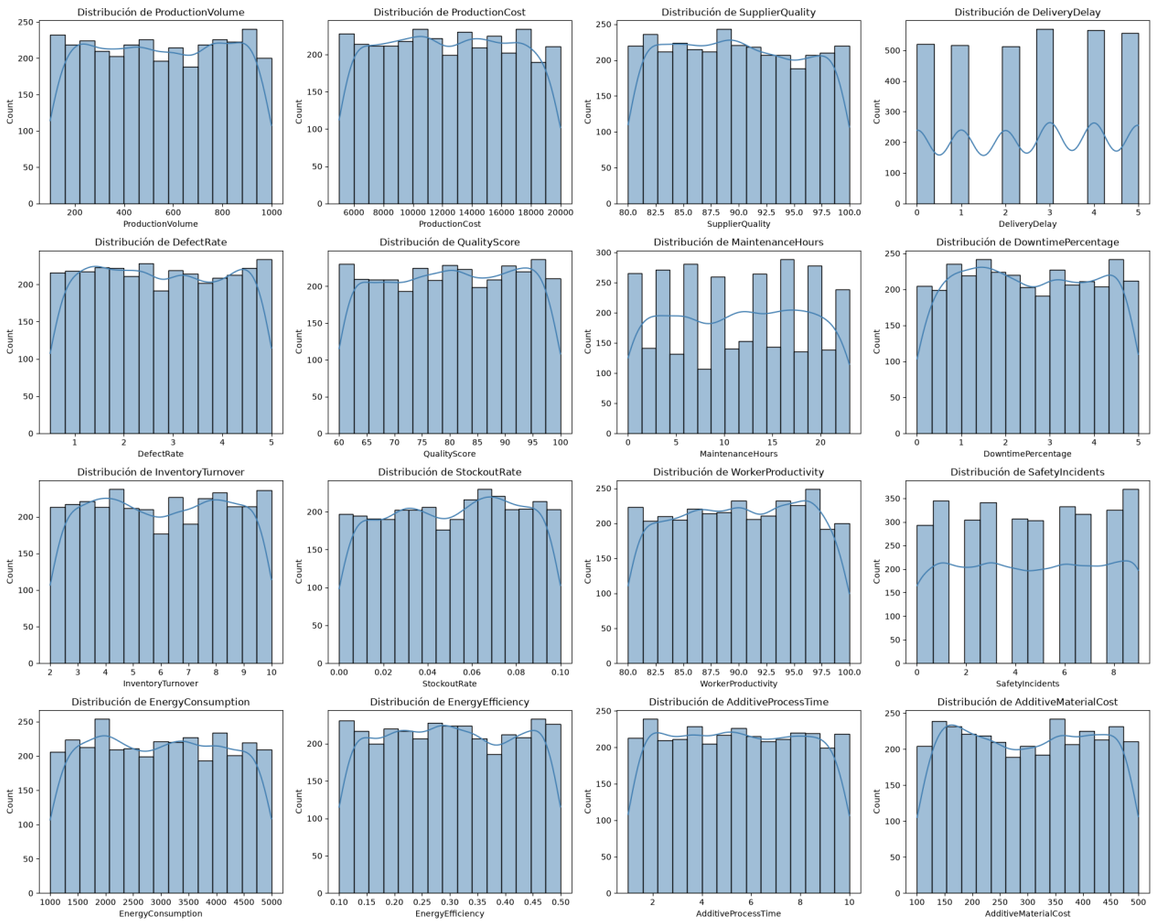
In [7]:

```
variables = data.columns.drop("DefectStatus")

fig, axes = plt.subplots(nrows=4, ncols=4, figsize=(20, 16))
axes = axes.flatten()

for i, col in enumerate(variables):
    sns.histplot(data[col], kde=True, ax=axes[i], color="steelblue")
    axes[i].set_title(f"Distribución de {col}")

plt.tight_layout()
plt.show()
```



In [8]:

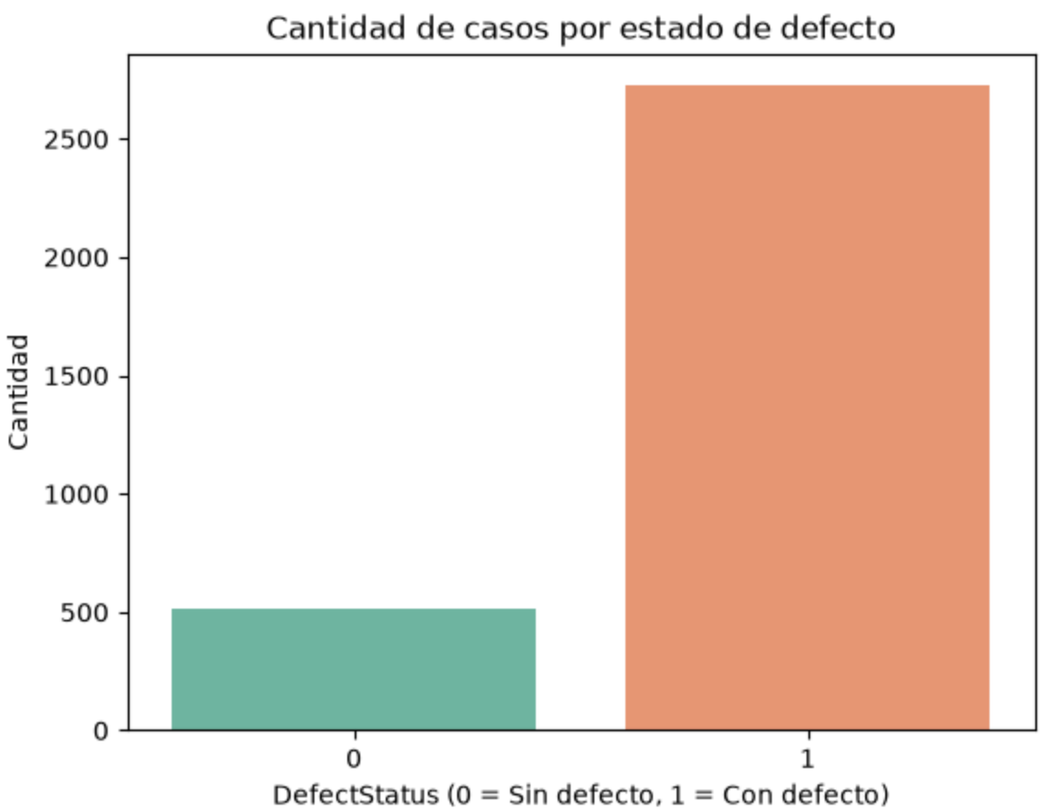
```
variables = data.columns.drop("DefectStatus")

fig, axes = plt.subplots(nrows=4, ncols=4, figsize=(20, 16))
axes = axes.flatten()

for i, col in enumerate(variables):
    sns.histplot(data[col], kde=True, ax=axes[i], color="steelblue")
    axes[i].set_title(f"Distribución de {col}")

plt.tight_layout()
plt.show()
```



In [9]:

```
conteo_defectos = data["DefectStatus"].value_counts()
print(conteo_defectos)

print("\nPorcentaje:")
print(data["DefectStatus"].value_counts(normalize=True) * 100)

# Gráfico
sns.countplot(x="DefectStatus", data=data, palette="Set2")
plt.title("Cantidad de casos por estado de defecto")
plt.xlabel("DefectStatus (0 = Sin defecto, 1 = Con defecto)")
plt.ylabel("Cantidad")
plt.show()
```

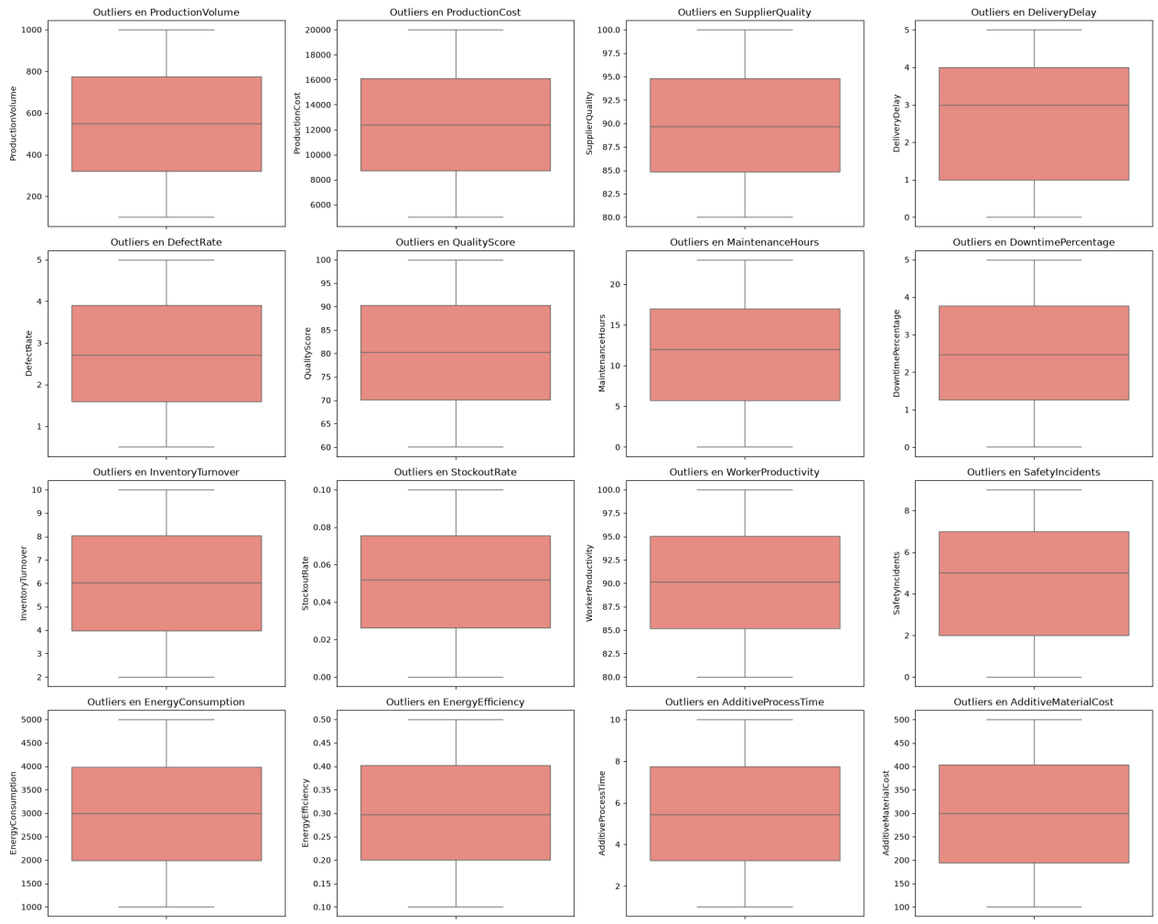
```
DefectStatus
1    2723
0     517
Name: count, dtype: int64

Porcentaje:
DefectStatus
1    84.04321
0    15.95679
Name: proportion, dtype: float64
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\2480138692.py:8: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x="DefectStatus", data=data, palette="Set2")
```



In [10]:

```
fig, axes = plt.subplots(nrows=4, ncols=4, figsize=(20, 16))
axes = axes.flatten()

for i, col in enumerate(variables):
    sns.boxplot(y=data[col], ax=axes[i], color="salmon")
    axes[i].set_title(f"Outliers en {col}")

plt.tight_layout()
plt.show()

# Método IQR para cuantificar outliers
print("Cantidad de outliers por variable (método IQR):\n")
for col in variables:
    Q1 = data[col].quantile(0.25)
    Q3 = data[col].quantile(0.75)
    IQR = Q3 - Q1
```

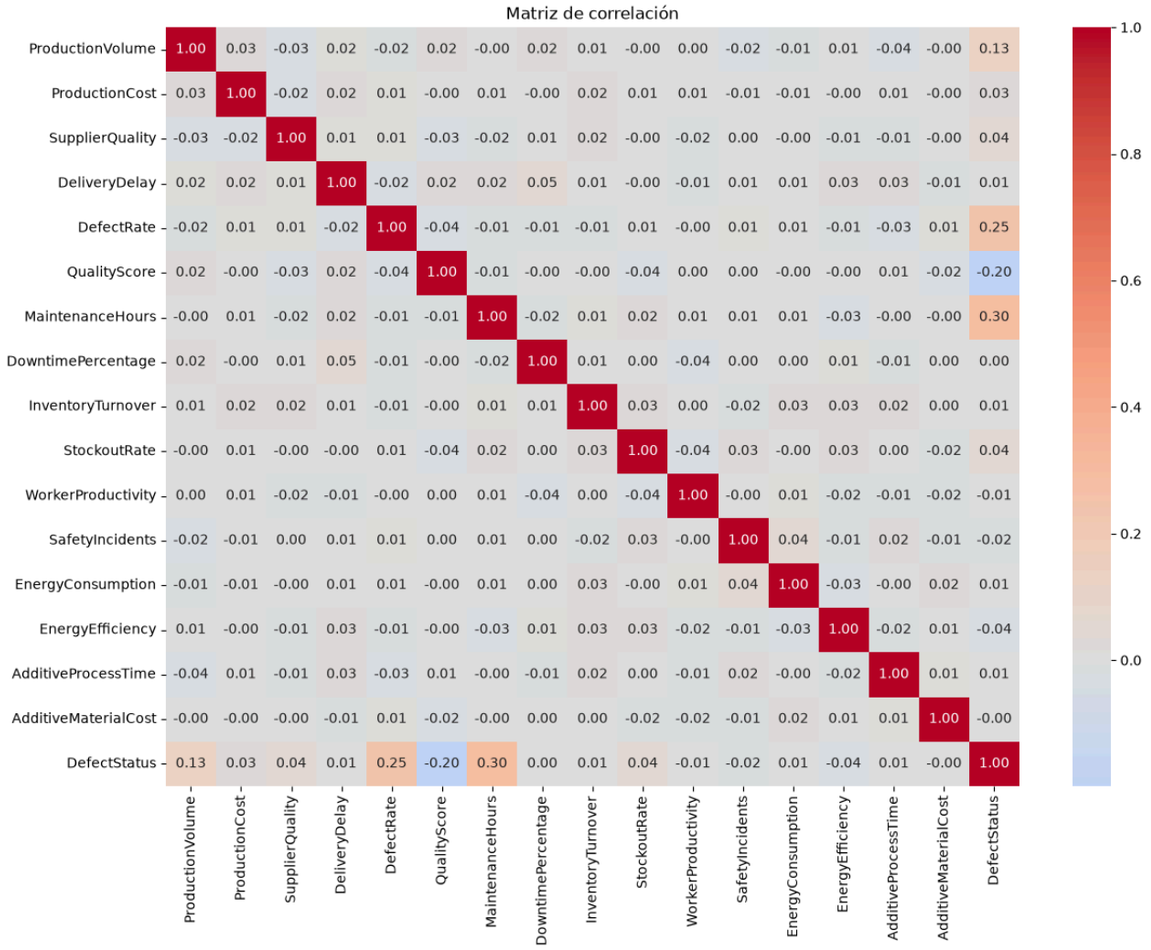
```
    limite_inferior = Q1 - 1.5 * IQR
    limite_superior = Q3 + 1.5 * IQR
    outliers = data[(data[col] < limite_inferior) | (data[col] > limite_superior)]
    print(f"{col}: {len(outliers)} outliers")
```




```
Cantidad de outliers por variable (método IQR):

ProductionVolume: 0 outliers
ProductionCost: 0 outliers
SupplierQuality: 0 outliers
DeliveryDelay: 0 outliers
DefectRate: 0 outliers
QualityScore: 0 outliers
MaintenanceHours: 0 outliers
DowntimePercentage: 0 outliers
InventoryTurnover: 0 outliers
StockoutRate: 0 outliers
WorkerProductivity: 0 outliers
SafetyIncidents: 0 outliers
EnergyConsumption: 0 outliers
EnergyEfficiency: 0 outliers
AdditiveProcessTime: 0 outliers
AdditiveMaterialCost: 0 outliers
```

In [11]:

```
plt.figure(figsize=(14, 10))
matriz_corr = data.corr()
sns.heatmap(matriz_corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Matriz de correlación")
plt.show()

# Correlación de cada variable con DefectStatus, ordenada
print("Correlación con DefectStatus:")
print(matriz_corr["DefectStatus"].sort_values(ascending=False))
```



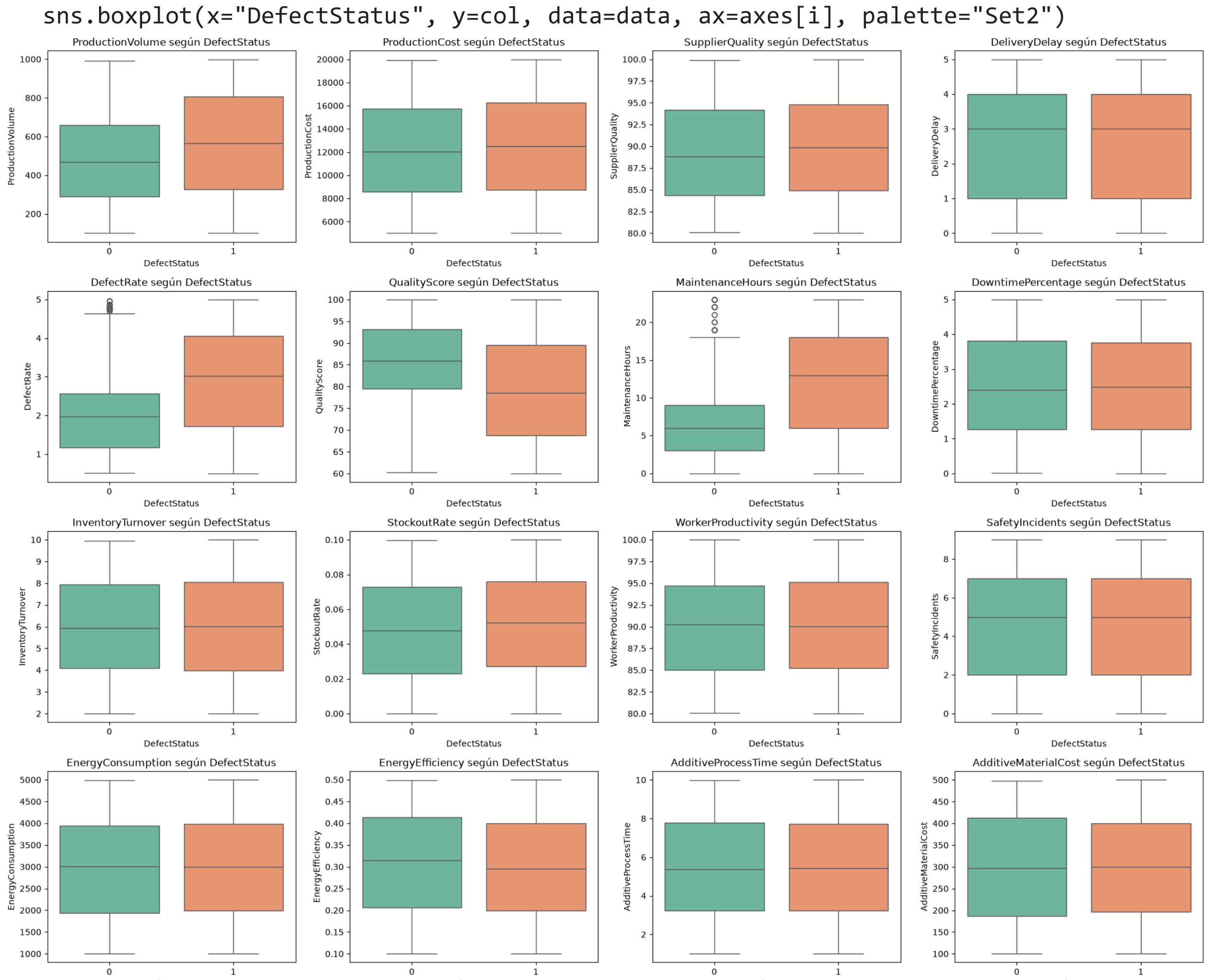

```
Correlación con DefectStatus:
DefectStatus             1.000000
MaintenanceHours         0.297107
DefectRate               0.245746
ProductionVolume         0.128973
StockoutRate             0.040574
SupplierQuality          0.038184
ProductionCost           0.026720
InventoryTurnover        0.006733
AdditiveProcessTime      0.005619
DeliveryDelay            0.005425
EnergyConsumption        0.005039
DowntimePercentage       0.004128
AdditiveMaterialCost    -0.000953
WorkerProductivity      -0.005224
SafetyIncidents         -0.016039
EnergyEfficiency        -0.035031
QualityScore            -0.199219
Name: DefectStatus, dtype: float64
```

In [12]:

```python
fig, axes = plt.subplots(nrows=4, ncols=4, figsize=(20, 16))
axes = axes.flatten()

for i, col in enumerate(variables):
    sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
    axes[i].set_title(f"{col} según DefectStatus")

plt.tight_layout()
plt.show()

# Tabla comparativa de medias por grupo
comparacion = data.groupby("DefectStatus")[variables].mean().T
comparacion.columns = ["Media (Sin defecto)", "Media (Con defecto)"]
comparacion["Diferencia"] = comparacion["Media (Con defecto)"] - comparacion["Media (
comparacion.sort_values("Diferencia", ascending=False)
```

```
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
```

```
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
C:\Users\salasistemas.UTB\AppData\Local\Temp\ipykernel_9548\1283225898.py:5: FutureWarn
ing:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="DefectStatus", y=col, data=data, ax=axes[i], palette="Set2")
```



Out[12]:

	Media (Sin defecto)	Media (Con defecto)	Diferencia
ProductionCost	12158.877326	12473.169403	314.292076
ProductionVolume	470.866538	563.267352	92.400814
EnergyConsumption	2975.158109	2991.026547	15.868438
MaintenanceHours	6.791103	12.366140	5.575038
DefectRate	2.010328	2.889386	0.879057
SupplierQuality	89.328686	89.929096	0.600410
InventoryTurnover	5.983667	6.026496	0.042829
AdditiveProcessTime	5.438599	5.478459	0.039860
DeliveryDelay	2.537718	2.562982	0.025264
DowntimePercentage	2.487697	2.503970	0.016273
StockoutRate	0.048197	0.051387	0.003190
EnergyEfficiency	0.309132	0.298000	-0.011133
WorkerProductivity	90.108731	90.027088	-0.081643
SafetyIncidents	4.698259	4.571429	-0.126831
AdditiveMaterialCost	299.769855	299.467182	-0.302673
QualityScore	85.442375	79.126454	-6.315922

In [13]:

```
print("""
CONCLUSIONES PRELIMINARES:

1. Calidad de datos:
   - El dataset tiene {filas} filas y {columnas} columnas.
   - {faltantes} valores faltantes y {duplicados} registros duplicados detectados.

2. Balance de clases:
   - {pct_0:.1f}% de los productos NO presentan defectos (DefectStatus = 0).
   - {pct_1:.1f}% de los productos SÍ presentan defectos (DefectStatus = 1).

3. Variables más asociadas al defecto (revisar la correlación más alta con DefectStat

4. Variables con más outliers pueden requerir tratamiento antes de modelar.

5. Las variables con mayor diferencia de medias entre grupos con y sin defecto
   son candidatas fuertes como predictoras en un modelo de clasificación.
""".format(
    filas=data.shape[0],
    columnas=data.shape[1],
    faltantes=data.isnull().sum().sum(),
    duplicados=data.duplicated().sum(),
    pct_0=(data["DefectStatus"].value_counts(normalize=True)*100)[0],
    pct_1=(data["DefectStatus"].value_counts(normalize=True)*100)[1]
))
```

```
CONCLUSIONES PRELIMINARES:

1. Calidad de datos:
   - El dataset tiene 3240 filas y 17 columnas.
   - 0 valores faltantes y 0 registros duplicados detectados.

2. Balance de clases:
   - 16.0% de los productos NO presentan defectos (DefectStatus = 0).
   - 84.0% de los productos SÍ presentan defectos (DefectStatus = 1).

3. Variables más asociadas al defecto (revisar la correlación más alta con DefectStatu
s).

4. Variables con más outliers pueden requerir tratamiento antes de modelar.

5. Las variables con mayor diferencia de medias entre grupos con y sin defecto
   son candidatas fuertes como predictoras en un modelo de clasificación.
```

In [16]:

```
# Exportar a pdf todo el notebook
!jupyter nbconvert --to pdf Tarea1.ipynb
```

```
[NbConvertApp] Converting notebook Tarea1.ipynb to pdf
[NbConvertApp] Support files will be in Tarea1_files\
[NbConvertApp] Making directory .\Tarea1_files
[NbConvertApp] Writing 59950 bytes to notebook.tex
[NbConvertApp] Building PDF
Traceback (most recent call last):
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Scripts\jupyter-nbcon
vert-script.py", line 9, in <module>
    sys.exit(main())
             ^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\jup
yter_core\application.py", line 284, in launch_instance
    super().launch_instance(argv=argv, **kwargs)
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\tra
itlets\config\application.py", line 1043, in launch_instance
    app.start()
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\nbconvertapp.py", line 420, in start
    self.convert_notebooks()
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\nbconvertapp.py", line 597, in convert_notebooks
    self.convert_single_notebook(notebook_filename)
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\nbconvertapp.py", line 563, in convert_single_notebook
    output, resources = self.export_single_notebook(
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\nbconvertapp.py", line 487, in export_single_notebook
    output, resources = self.exporter.from_filename(
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\templateexporter.py", line 390, in from_filename
    return super().from_filename(filename, resources, **kw)  # type:ignore[return-valu
e]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\exporter.py", line 201, in from_filename
    return self.from_file(f, resources=resources, **kw)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\templateexporter.py", line 396, in from_file
    return super().from_file(file_stream, resources, **kw)  # type:ignore[return-value]
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\exporter.py", line 220, in from_file
    return self.from_notebook_node(
           ^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\pdf.py", line 197, in from_notebook_node
    self.run_latex(tex_file)
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\pdf.py", line 166, in run_latex
    return self.run_command(
           ^^^^^^^^^^^^^^^^^
  File "C:\Users\salasistemas.UTB\.conda\envs\ml_advanced_project\Lib\site-packages\nbc
onvert\exporters\pdf.py", line 120, in run_command
    raise OSError(msg)
OSError: xelatex not found on PATH, if you have not installed xelatex you may need to d
o so. Find further instructions at https://nbconvert.readthedocs.io/en/latest/install.h
tml#installing-tex.
```